

Quadruped Studio

Desktop application architecture for developing, simulating, configuring and controlling multiple robot dogs

Scope: evolve the proven current-dog interface into a reusable desktop tool supporting multiple quadrupeds, project files, URDF/model import, Isaac Sim workflows, hardware drivers, simulation backends and future policy deployment.

Robot Dog Software Roadmap

13 August 2026

Document map

This document is intended to be used as a build roadmap: start at the top, implement the high-value foundations first, and leave the speculative features until the core robot workflow is reliable.

- 1. Product vision** What the desktop application is for and why it is worth building.
- 2. Core principles** Rules that keep the project reusable without becoming a replacement for Isaac Sim.
- 3. Desktop architecture** Frontend, robot model, kinematics, transports, simulation and native services.
- 4. Robot project format** How a dog is stored as data rather than hard-coded behaviour.
- 5. Current dog as the reference robot** How to migrate the existing dog without blocking on generic features.
- 6. Import and setup workflow** URDF, GLB/GLTF, CAD-assisted workflows and the rigging wizard.
- 7. Isaac Sim integration** File interchange, ROS 2 live simulation, and a future Isaac extension.
- 8. Kinematics and gait system** Reusable solvers, gait definitions and visual validation.
- 9. Hardware driver layer** ESP32 first, then serial, UDP, ROS 2 and other controllers.
- 10. Simulation backends** Built-in preview, Isaac Sim and physical-robot targets under one command interface.
- 11. Desktop-only capabilities** Filesystem, native processes, logging, firmware tools and performance-sensitive services.
- 12. User experience** Project home, import wizard, robot workspace and development tools.
- 13. Development with Codex** How the codebase and build loop should work during active development.
- 14. Roadmap** A staged build from packaged current dog to a reusable platform.
- 15. Risks and non-goals** Boundaries that keep the idea achievable.
- 16. Technical references** Current NVIDIA Isaac Sim interoperability points used by this proposal.

1. Product vision

Quadruped Studio should be a desktop development environment for robot dogs: import or define a quadruped, inspect its structure, tune kinematics and gaits, simulate it, connect real hardware, calibrate actuators and observe the entire control stack.

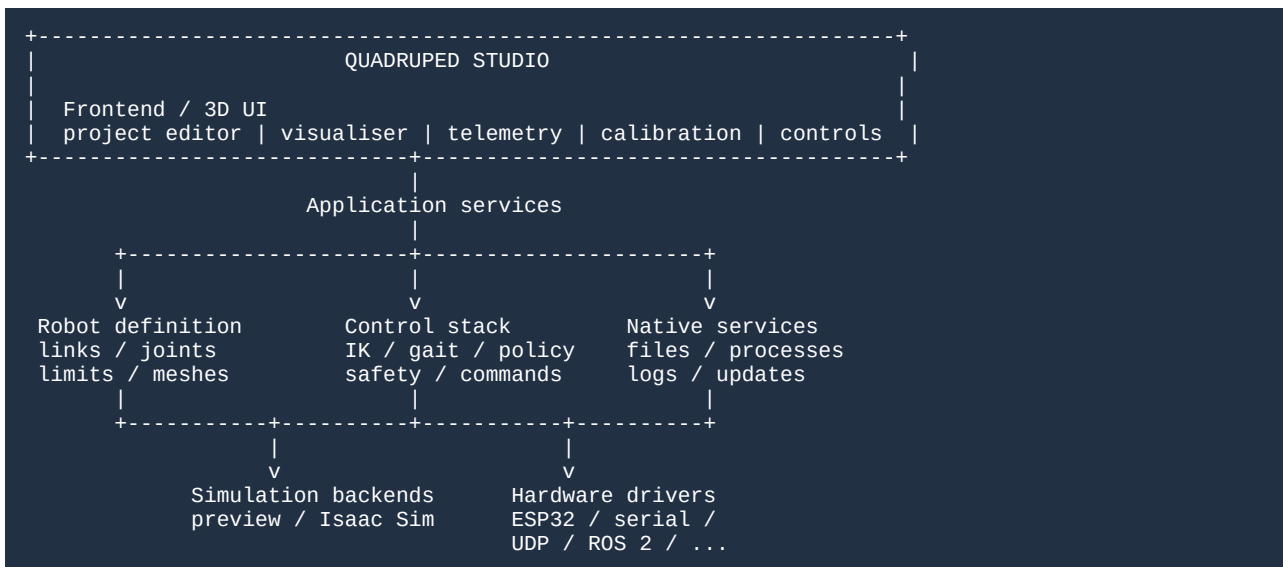
The value is not simply packaging a website into an .exe. The desktop application becomes the place where a robot project lives. It can combine the existing interactive 3D frontend with native file access, local services, external simulation, robot drivers, configuration versioning and development tools.

The current dog remains the first-class reference implementation. It should work extremely well before the platform claims broad support. Generic features are added by extracting stable interfaces from real requirements rather than guessing what every quadruped will need.

2. Core principles

Principle	Meaning
Project-driven	Robot dimensions, joint mapping, meshes, limits, calibration and drivers are data stored in a project.
Quadruped-specific	Support robot dogs well; do not immediately expand into arms, humanoids, drones and every URDF robot.
One control model	The same high-level command can target preview, Isaac Sim or physical hardware.
Simulation is external when appropriate	Use Isaac Sim for serious physics rather than re-implementing a full simulator.
Visual first	Users should configure joints, axes and mappings through the 3D viewport wherever possible.
Safe by default	Imported or newly configured robots start disarmed with conservative limits.
Extensible by interfaces	Add drivers, solvers and importers behind stable APIs rather than branching the UI for each robot.

3. Desktop architecture



3.1 Recommended packaging approach

Keep the existing web frontend. Package it as a desktop application and add a native backend only where the desktop environment provides real value. A Tauri-style architecture is a strong fit: web UI and 3D rendering remain familiar, while native commands handle files, processes, device integration and performance-sensitive work. Electron is also viable if development simplicity outweighs memory footprint.

Performance expectation

Packaging the frontend does not automatically make WebGL/WebGPU rendering faster; the browser already uses the computer GPU. The major new capability is access to native services and the option to move heavy computation or device integration out of browser JavaScript.

3.2 Stable internal interfaces

RobotDefinition	KinematicsSolver	RobotDriver
- links	- solveIK()	- connect()
- joints	- solveFK()	- disconnect()
- limits	- validateTarget()	- sendCommand()
- geometry		- readTelemetry()
- sensors		
- actuators	GaitController	SimulationBackend
	- update()	- loadRobot()
Project	- presets	- reset()
- robot	- phase state	- step()
- calibration		- getState()
- controller		
- simulation		
- UI preferences		

4. Robot project format

The key shift from the current browser app is that a robot becomes a project on disk. The application loads the project and configures itself around the robot definition.

```

MyDog.qstudio/
|- project.json
|- robot/
|  |- robot.json
|  |- robot.urdf
|  |- visual.glb
|  `-- meshes/
|- calibration/
|  `-- current.json
|- gaits/
|  |- walk.json
|  `-- trot.json
|- controller/
|  `-- gamepad.json
|- simulation/
|  `-- isaac.json
|- logs/
`-- policies/

```

A folder-based project is easier to inspect, version in Git and repair manually than a monolithic binary file. A later packaged extension such as .qdog or .qstudio can simply be a zip container around this structure.

4.1 Robot definition

```

{
  "name": "Dog V2",
  "type": "quadruped",
  "links": [...],
  "joints": [...],
  "legMapping": {
    "frontLeft": ["FL_HAA", "FL_HFE", "FL_KFE"]
  },
  "kinematics": { "solver": "generic-3dof" },
  "driver": { "type": "esp32", "profile": "dog-v2" }
}

```

- Use SI units internally: metres, radians, kilograms and seconds.
- Give every link and joint a stable ID independent of display name.
- Store coordinate-frame conventions explicitly.
- Separate mechanical joint limits from user-configured safety limits.
- Keep actuator mapping separate from the kinematic joint hierarchy.
- Version the schema so old robot projects can be migrated safely.

5. Current dog as the reference robot

Do not require the current dog to go through a complex import wizard. Ship a built-in “Dog V2” project that mirrors the existing browser environment and hardware stack. The first desktop milestone is feature parity with the browser plus project persistence and native packaging.

Existing capability	Desktop representation
Current 3D model	robot/visual.glb or equivalent project asset.
Current link dimensions	RobotDefinition geometry and joint origins.
Current servo mappings	ESP32 driver profile + calibration file.
Current controller mapping	Controller profile.
Current gait constants	Editable gait preset files.

Existing capability	Desktop representation
Current telemetry	Common telemetry schema used by dashboard and logs.

Migration rule

The desktop application should be able to open the current dog and immediately behave like the finished browser version. Generalisation is successful only if it does not make the reference robot harder to use.

6. Import and setup workflow

6.1 Recommended import order

Stage	Format	Role
V1	URDF + referenced meshes	Primary structured robot import: links, joints, axes, limits and visuals.
V1	GLB / GLTF	Visual model import or replacement when a structured robot definition already exists.
V2	USD via Isaac workflow	Interchange through Isaac Sim, initially by exporting URDF from USD.
V2/V3	Xacro via ROS 2 / conversion	Useful for ROS-based robot descriptions.
Later	STEP / CAD assembly	Assisted geometry import, but still requires semantic joint setup.

URDF should be the first serious import format because it carries robot semantics. A raw CAD model can show geometry, but it does not reliably tell the application which pieces are actuated, their motion axes, control meaning or safety limits.

6.2 Visual setup wizard

1. Import the robot description and meshes.
2. Show the detected link/joint tree and highlight each item in the viewport.
3. Classify the robot as a quadruped and map the four legs.
4. Map each leg joint to semantic roles such as hip abduction, hip flexion and knee flexion.
5. Verify joint axes and positive rotation directions using an interactive gizmo.
6. Set or confirm mechanical and safe operating limits.
7. Choose a kinematics solver and validate reachable foot workspace.
8. Choose a hardware driver or simulation-only mode.
9. Run a neutral-pose validation before allowing the project to arm physical hardware.

6.3 CAD-assisted import

CAD upload is still valuable, but it should be treated as a geometry source. If the imported assembly contains a useful component hierarchy, the wizard can use that hierarchy as a starting point; the user still

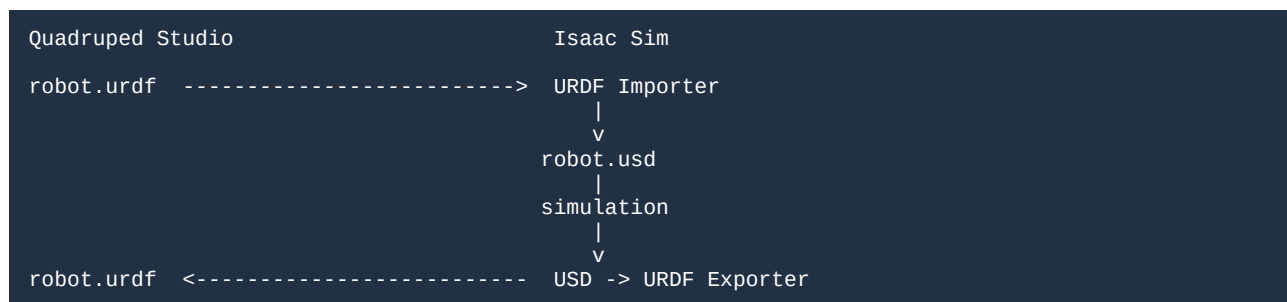
confirms joints, axes and semantic mappings. This keeps the feature realistic instead of claiming arbitrary CAD can be automatically understood as a robot.

7. Isaac Sim integration

Isaac Sim should be treated as a simulation partner and interchange target, not something Quadruped Studio attempts to replace.

Current NVIDIA Isaac Sim documentation provides a URDF importer that converts URDF robot descriptions to USD, a USD-to-URDF exporter, and a ROS 2 bridge for bidirectional communication between Isaac Sim and ROS 2 systems. Those three capabilities create a practical staged integration path.

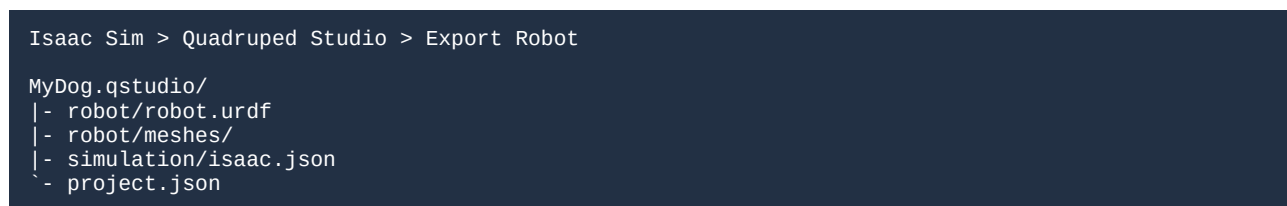
7.1 Level 1 - file interchange



- Import a robot configured in Isaac Sim by exporting it to URDF and opening the resulting URDF + meshes in Quadruped Studio.
- Export a Quadruped Studio robot as URDF so it can be imported into Isaac Sim and converted to USD.
- Keep Quadruped Studio-only data - controller mapping, gait presets, hardware calibration - in the project rather than forcing it into URDF.
- Add an import report that lists unsupported or lossy properties instead of silently ignoring them.

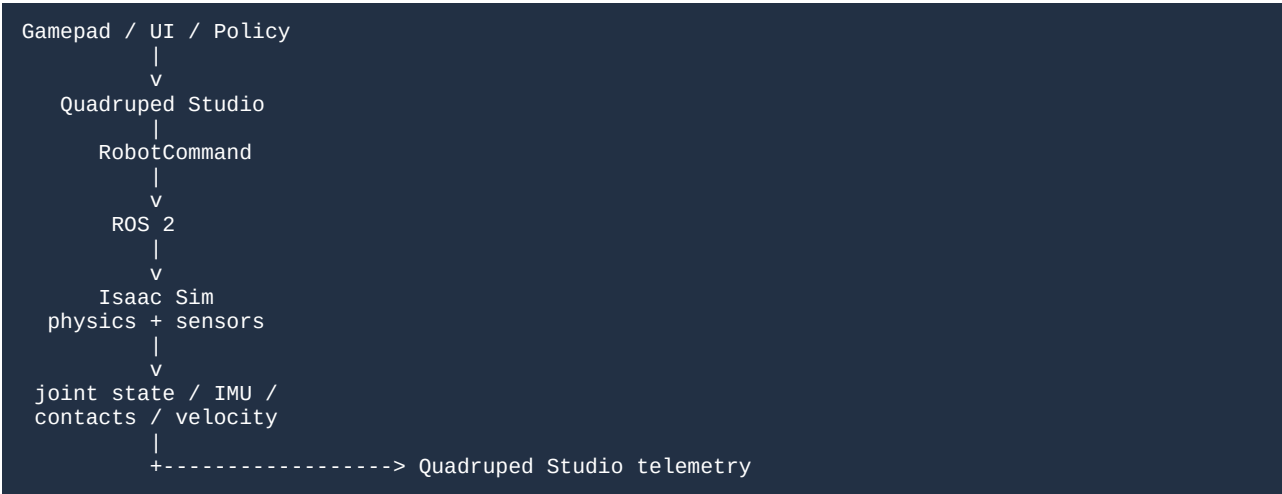
7.2 Level 2 - Quadruped Studio Isaac extension

A later Isaac Sim extension could export a project bundle directly. That extension can inspect the selected articulation, export a compatible robot description and attach Quadruped Studio metadata such as the four-leg semantic mapping.



7.3 Level 3 - live simulation backend

The strongest long-term integration is a live backend where Quadruped Studio sends commands to Isaac Sim and receives joint, IMU, contact and other simulated state back. ROS 2 is a natural initial transport because Isaac Sim provides a dedicated ROS 2 bridge.



Boundary

Do not require Isaac Sim to be installed for normal use. The application should always retain a lightweight built-in kinematic preview. Isaac becomes an optional high-fidelity backend.

7.4 Isaac project metadata

- Isaac asset path or generated USD path.
- ROS 2 namespace and topic mapping.
- Simulation timestep and expected control rate.
- Joint command interface type.
- Sensor topic mapping for IMU, contacts and cameras.
- Transform/frame mapping where names differ from the local robot project.

8. Kinematics and gait system

Generic support does not mean one giant solver that handles every possible leg. Solvers should be plugins selected by the robot definition. Start with the exact 3-DOF leg topology used by the current dog, then add alternative topologies only when there is a real target robot.



Layer	Responsibility
Kinematics solver	Transforms body/foot targets into joint targets and reports invalid/unreachable requests.
Gait generator	Produces time-varying foot targets and body targets.

Layer	Responsibility
Safety limiter	Constrains joint, velocity and body targets to configured safe ranges.
Actuator mapping	Converts semantic joint targets into hardware-specific output values.

8.1 Gaits as data

```
{
  "name": "Trot",
  "phaseOffsets": {"FL": 0.0, "FR": 0.5, "RL": 0.5, "RR": 0.0},
  "dutyFactor": 0.55,
  "stepHeight": 0.060,
  "stepLength": 0.120,
  "cycleTime": 0.240
}
```

Storing gait presets as data makes them portable between compatible robots and versionable in Git. Robot-specific constraints can scale or clamp those values based on geometry and safety limits.

9. Hardware driver layer

A RobotDriver should make the rest of the application indifferent to whether the current dog is reached over Wi-Fi, serial, ROS 2 or another controller.

```
RobotDriver
|- connect()
|- disconnect()
|- arm()
|- disarm()
|- sendJointTargets()
|- sendBodyCommand()
|- readTelemetry()
|- getCapabilities()
|- getFirmwareInfo()
```

Driver	Priority	Notes
ESP32 Dog V2	First	Reference implementation; preserve current packet protocol initially.
Generic serial	Early	Useful for microcontroller robots and bench testing.
UDP / WebSocket	Early	Natural for Wi-Fi robots and remote controllers.
ROS 2	Medium	Useful for Isaac Sim, Jetson and ROS-native robots.

Driver	Priority	Notes
Vendor-specific SDK	Later	Only add when a real supported robot requires it.

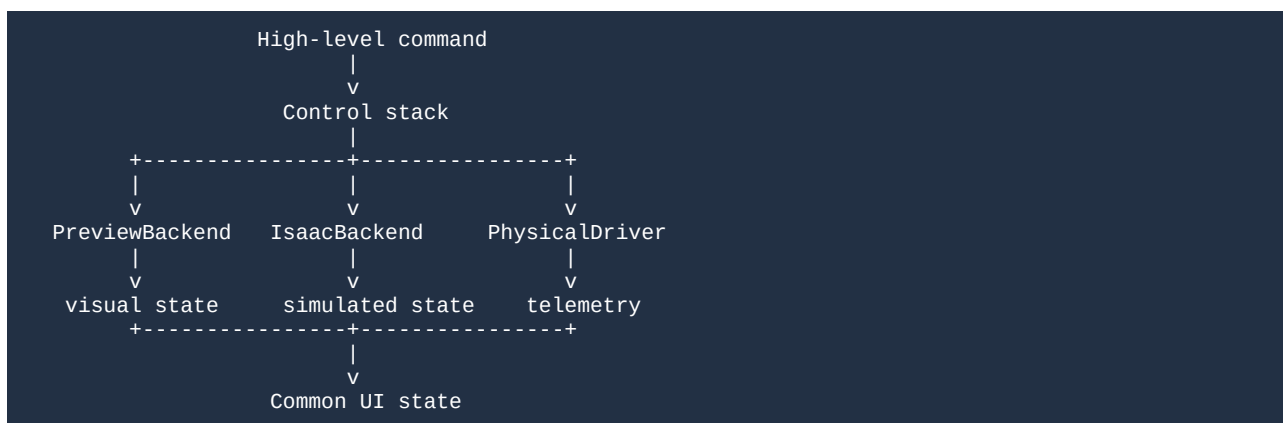
9.1 Capability negotiation

Drivers should report what the connected system can actually do. The UI can then hide unsupported controls instead of assuming every robot has voltage sensing, individual servo telemetry, firmware flashing or an IMU.

```
Capabilities
batteryTelemetry: true
imu: true
jointFeedback: false
servoTemperature: false
firmwareUpdate: true
velocityCommand: true
jointCommand: true
```

10. Simulation backends

Backend	Purpose	Expected fidelity
Built-in preview	Instant kinematic visualisation, IK validation, gait editing.	Low physics; always available.
Isaac Sim	Contact physics, sensors, terrain, policy development, realistic testing.	High; optional external dependency.
Physical robot	Real hardware execution.	Ground truth; constrained by safety and communications.



11. Desktop-only capabilities

These are the reasons to ship a desktop application rather than merely opening the existing site in a browser tab.

- Project folders and reliable local file access.

- Native serial/USB/Bluetooth integrations where browser APIs are limiting or inconsistent.
- Launch and manage local helper processes such as ROS 2 bridges, Python tools or conversion scripts.
- Persist large logs and test datasets without relying on browser storage.
- Bundle robot profiles, models and calibration backups.
- Integrate firmware flashing and version checks later.
- Use native Rust/C++/Python services for compute-heavy processing when profiling shows a real bottleneck.
- Create installers, desktop shortcuts, file associations and application updates.

11.1 Performance strategy

Workload	Best initial location
3D visualisation	Existing web frontend using GPU-accelerated browser/webview rendering.
UI state and controls	Frontend TypeScript/JavaScript.
Packet transport	Frontend or native service depending on required device API.
IK/gait math	Keep where it is until profiling shows it is a bottleneck.
Large log processing	Native/background service.
Policy inference	Dedicated runtime on PC/Jetson depending on model and deployment target.
Isaac physics	Isaac Sim, not reimplemented in the desktop app.

12. User experience

12.1 Home screen

QUADRUPEL STUDIO	
Recent Robots	
+-----+ +-----+	
Dog V2 SpotMicro Test	
12 DOF - ESP32 12 DOF - Simulation	
Open Open	
+-----+ +-----+	
[+ New robot] [Import URDF] [Open project]	

12.2 Robot workspace

+-----+ +-----+ +-----+ +-----+ +-----+	
MyDog SIM: ISAAC DRIVER: ESP32 CONNECTED DISARMED	
+-----+ +-----+ +-----+ +-----+ +-----+	
Project	Inspector
Control	selected joint
Calib	limits
Gaits	telemetry
Sim	mapping
Logs	
+-----+ +-----+ +-----+ +-----+ +-----+	
timeline / controller / command scope	
+-----+ +-----+ +-----+ +-----+ +-----+	

12.3 Import wizard experience

1. Choose source: URDF, model-only, existing project or Isaac-exported bundle.
2. Inspect imported link/joint tree.
3. Map four legs and semantic joint roles.
4. Check axes, joint limits and neutral pose in 3D.
5. Choose solver and run workspace validation.
6. Choose simulation and hardware targets.
7. Save as a project and enter the normal workspace.

13. Development with Codex

Keep the repository structured so most feature changes remain ordinary frontend development. The desktop wrapper and native backend should be thin until a feature genuinely requires them.

```
quadruped-studio/
|- apps/
|  |- desktop/
|  `-- web-preview/
|- packages/
|  |- ui/
|  |- robot-core/
|  |- kinematics/
|  |- gait/
|  |- project-format/
|  `-- drivers/
|- native/
|  `-- desktop-backend/
|- robots/
|  `-- dog-v2/
```

13.1 Normal development loop

```
Codex edits frontend / core package
  |
  v
dev server rebuild
  |
  v
hot reload / refresh
  |
  v
test change

Native backend change
  |
  v
restart/rebuild native dev process
  |
  v
test change
```

- Most layout, 3D, telemetry and project-editor changes should hot reload.
- Native process restarts are expected only when changing native commands, permissions or packaging configuration.
- Release installers are built periodically, not after every edit.
- Automated schema validation, importer fixtures and simulated driver tests are especially valuable for Codex-assisted development because they prevent “generic” changes from silently breaking the reference dog.

14. Roadmap

Version	Goal	Major deliverables
V0	Finish browser reference	Telemetry, safety, calibration, controller diagnostics, gait tuning, logs.
V1	Desktop Dog V2	Desktop package, project folder, current-dog parity, native file/log services.
V1.5	Config-driven robot	Move geometry, joints, calibration, gait presets and drivers into project data.
V2	Generic 3-DOF quadrupeds	URDF import, leg mapping wizard, generic 3-DOF solver, multiple projects.
V2.5	Isaac interchange	Export/import URDF workflow, Isaac project metadata, validation reports.
V3	Live Isaac backend	ROS 2 integration, simulated state/command mapping, Isaac launch/connect tooling.
V3.5	Driver ecosystem	ROS 2 driver, generic serial/UDP profiles, capability negotiation.
V4	Policy workflow	Policy packages, simulation testing, deployment target abstraction.

14.1 Minimum viable generic release

A credible first public “generic” release does not need arbitrary CAD intelligence or every dog on the market.

- Windows desktop build.
- Open/save multiple robot projects.
- Built-in current Dog V2 template.
- URDF import with meshes.
- Four-leg / three-joint semantic mapping wizard.
- Generic 3-DOF quadruped IK.
- Built-in kinematic preview.
- At least one generic transport plus the Dog V2 ESP32 driver.
- Calibration, controller, telemetry and gait tools from the browser version.
- URDF export that can be consumed by Isaac Sim.

15. Risks and non-goals

Risk	Control
Trying to support every robot type	Keep the product explicitly quadruped-focused until the architecture is proven.
Arbitrary CAD auto-rigging becomes a research project	Treat CAD as geometry and use an assisted joint/axis wizard.
Rebuilding Isaac Sim badly	Keep built-in simulation kinematic/lightweight; delegate serious physics to Isaac.
Importer complexity and format loss	Generate clear import reports, preserve source assets and make mappings editable.
Generic architecture slows current dog	Use Dog V2 as the regression/reference project for every major change.
Unsafe imported limits or mappings	Disarmed default, validation checks, conservative initial limits and neutral-pose preview.
Plugin architecture too early	Start with internal interfaces; expose third-party plugins only after multiple real implementations exist.

15.1 Explicit non-goals for early versions

- General-purpose robotics IDE for arms, humanoids and wheeled robots.
- Full rigid-body physics engine competitive with Isaac Sim.
- Automatic understanding of arbitrary STEP assemblies with no user setup.
- Cloud account/platform dependency.
- Support for every vendor protocol before there is an actual user/robot requiring it.

16. Technical references

The Isaac Sim portions of this roadmap are based on current NVIDIA documentation checked on 13 August 2026. The precise UI and extension names may evolve, so integration code should target documented APIs and validate the installed Isaac Sim version.

Reference	Relevance
NVIDIA Isaac Sim - Tutorial: Import URDF	Documents importing URDF robot descriptions and converting them to USD; includes GUI and Python workflows.
NVIDIA Isaac Sim - Tutorial: Export URDF	Documents the USD-to-URDF exporter and resulting URDF + mesh output.
NVIDIA Isaac Sim - ROS 2 Bridge	Documents bidirectional ROS 2 integration used as the proposed live simulation transport.

URLs:

- https://docs.isaacsim.omniverse.nvidia.com/latest/importer_exporter/import_urdf.html

- https://docs.isaacsim.omniverse.nvidia.com/latest/importer_exporter/export_urdf.html
- <https://docs.isaacsim.omniverse.nvidia.com/latest/py/source/extensions/isaacsim.ros2.bridge/docs/index.html>

Recommended long-term endpoint

Quadruped Studio succeeds when one interface can open the current dog, import another quadruped, validate its joints, preview or simulate motion, map hardware, calibrate it, and send the same control stack to either Isaac Sim or the physical robot.

The most important near-term architectural decision is therefore small: finish the current dog cleanly, but store new robot-critical information - calibration, geometry, limits, mappings and gait parameters - in explicit data structures rather than burying more constants in UI and firmware code. That preserves the path from a good single-dog tool to a genuine quadruped development platform.